



BATCH AND ROLL NO:

EXPERIMENT NO.2

TITLE: Designing of mobile application to create home page using grid layout.

DATE OF PERFORMANCE

DATE OF CHECKING

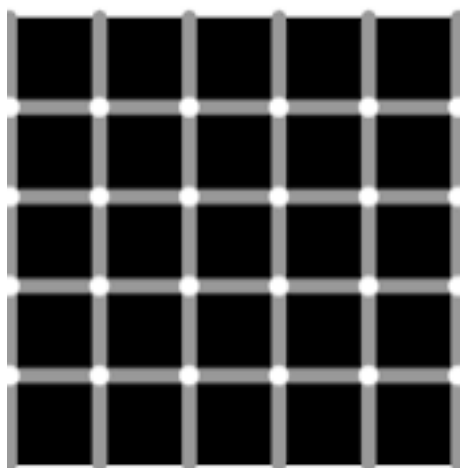
Title: Designing of mobile application to create home page using grid layout.

Requirements:

1. Android studio
2. Java SDK

Theory:

Grid is merely a skeleton that helps arrange the items on page. Grid Layout is a layout manager that arranges the views in a grid. In Android GridLayout, we can specify the number of columns and rows that the grid will have. We can customize the GridLayout according to our requirements, like setting the size, color or the margin for the Layout. GridLayout basically places its children in a rectangular grid. This grid has a set of a number of thin lines that separate the view area into cells. Suppose you have a grid of N columns, then we will have N+1 grid indices that would be starting from 0.





Specifications of Android GridLayout

1. Row and Columns

The children can occupy the cells in contiguous cells, by defining them as follows:

For rows:

This defines the number of rows that would be occupied and how the children would aligned within the group of cells.

`GridLayout.LayoutParams#rowSpec`

For Columns:

This defines the number of columns that would be occupied and how the children would be aligned within the group of cells.

`GridLayout.LayoutParams#columnSpec`

2. Default Cell Assignment

If children do not have specified rows and columns then, the cell location is assigned automatically using the following:

- `GridLayout#setOrientation(int)`
- `GridLayout#setRowCount(int)`
- `GridLayout#setColumnCount(int)`

Attributes of Android GridLayout:

The Specifications here describes the characteristics of the rows and columns of a group of cells. For a description of the characteristics of a GridLayout, we can use the following XML Attributes:

- **android:alignmentMode** – It sets the alignment of the margins and the boundaries.
- **android:columnCount** – It sets the maximum number of columns to create when positioning children automatically.
- **android:columnOrderPreserved** – It ensures that the column boundaries appear in the same order as its column indices.
- **android:rowCount** – It set the maximum no. of rows to create when positioning children automatically.
- **android:rowOrderPreserved** -It ensures that the row boundaries appear in the same order as its row indices.
- **android:useDefaultMargins** – It tells the GridLayout to use margins if no margin is specified in the parameter. For this, it needs to be set on True.



Steps:

1. Create a New project in Android studio.
 2. Add required dependencies.
 3. Working with XML file
 4. Open activity_main.XML file, which represents the UI of the project.
- Comments are added inside the code according to grid layout creation.

Android layout commands

<androidx.constraintlayout.widget.ConstraintLayout

```
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".MainActivity">
```

<GridView

```
android:id="@+id/idGVcourses"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:horizontalSpacing="6dp"
android:numColumns="2"
android:verticalSpacing="6dp" />
```

</androidx.constraintlayout.widget.ConstraintLayout>

Code:

JAVA CODE:-

MainActivity.java:

```
package com.example.gridlayoutapp;
import android.os.Bundle;
import android.widget.GridView;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
    GridView gridView;
    int[] images = {
        R.drawable.img1,
        R.drawable.img2,
```

```

R.drawable.img3,
    R.drawable.img4,
    R.drawable.img5
};
String[] names = {
    "Roll no.1",
    "Roll no.2",
    "Roll no.3",
    "Roll no.4",
    "Roll no.5"
};
@Override
protected void onCreate(Bundle
savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    gridView =
findViewById(R.id.gridView);
    GridAdapter adapter = new
GridAdapter(this, images, names);
    gridView.setAdapter(adapter);
}
}

```

GridAdapter.java:

```
package com.example.gridlayoutapp;
```

```

import android.content.Context;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.BaseAdapter;
import android.widget.ImageView;
import android.widget.TextView;

```

```

public class GridAdapter extends
BaseAdapter {

```

```

    Context context;
    int[] images;
    String[] names;
    LayoutInflater inflater;

```

```

    public GridAdapter(Context context, int[]
images, String[] names) {
        this.context = context;
        this.images = images;
        this.names = names;
        inflater = LayoutInflater.from(context);
    }

```

```

    @Override
    public int getCount() {
        return names.length;
    }

```

```

    @Override

```

```

    public Object getItem(int i) {
        return names[i];
    }

```

```

    @Override
    public long getItemId(int i) {
        return i;
    }

```

```

    @Override
    public View getView(int i, View view,
ViewGroup parent) {
        if (view == null) {
            view =
inflater.inflate(R.layout.grid_item, parent,
false);
        }

```

```

            ImageView imageView =
view.findViewById(R.id.imageView);
            TextView textView =
view.findViewById(R.id.textView);

```

```

            imageView.setImageResource(images[i]);
            textView.setText(names[i]);

```

```

            return view;
        }
    }
}

```

XML Code:-

activity_main.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.Constraint
```

```
Layout
xmlns:android="http://schemas.android.com/a
pk/res/android"
```

```
xmlns:app="http://schemas.android.com/apk/r
es-auto"
```

```
    android:layout_width="match_parent"
    android:layout_height="match_parent">
```

```
    <GridView
        android:id="@+id/gridView"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:numColumns="2"
        android:horizontalSpacing="10dp"
        android:verticalSpacing="10dp"
        android:padding="10dp"
        android:stretchMode="columnWidth"/>
```

```
</androidx.constraintlayout.widget.Constraint
Layout>
```

grid_item.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="10dp"
    android:background="@android:color/white"
    android:elevation="4dp">
```

```
    <ImageView
        android:id="@+id/imageView"
        android:layout_width="120dp"
        android:layout_height="120dp"
        android:layout_gravity="center"
        android:scaleType="centerCrop"/>
```

```
    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Roll no"
        android:textSize="16sp"
        android:layout_gravity="center"
        android:paddingTop="8dp"/>
```

```
</LinearLayout>
```



PUNE INSTITUTE OF COMPUTER TECHNOLOGY, PUNE – 411043
Department of Electronics & Telecommunication Engineering

Output:

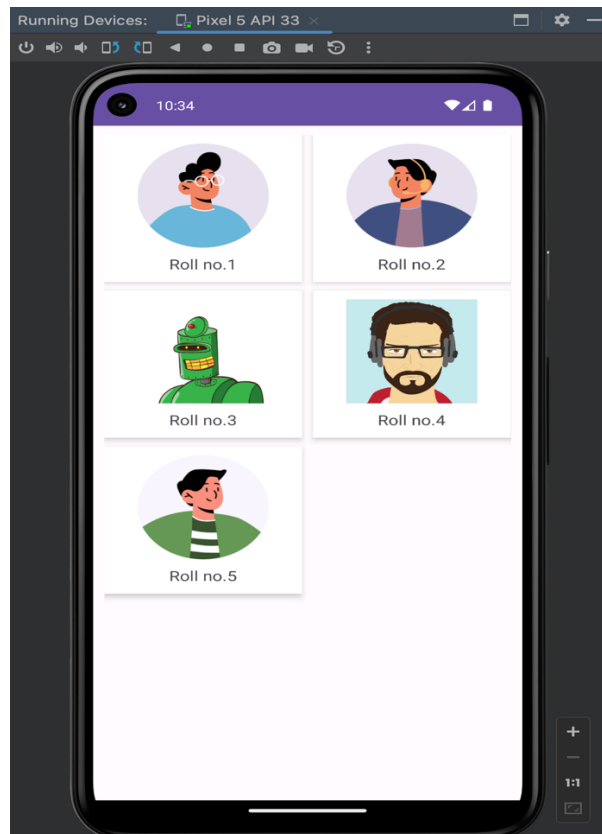


fig a. Grid layout

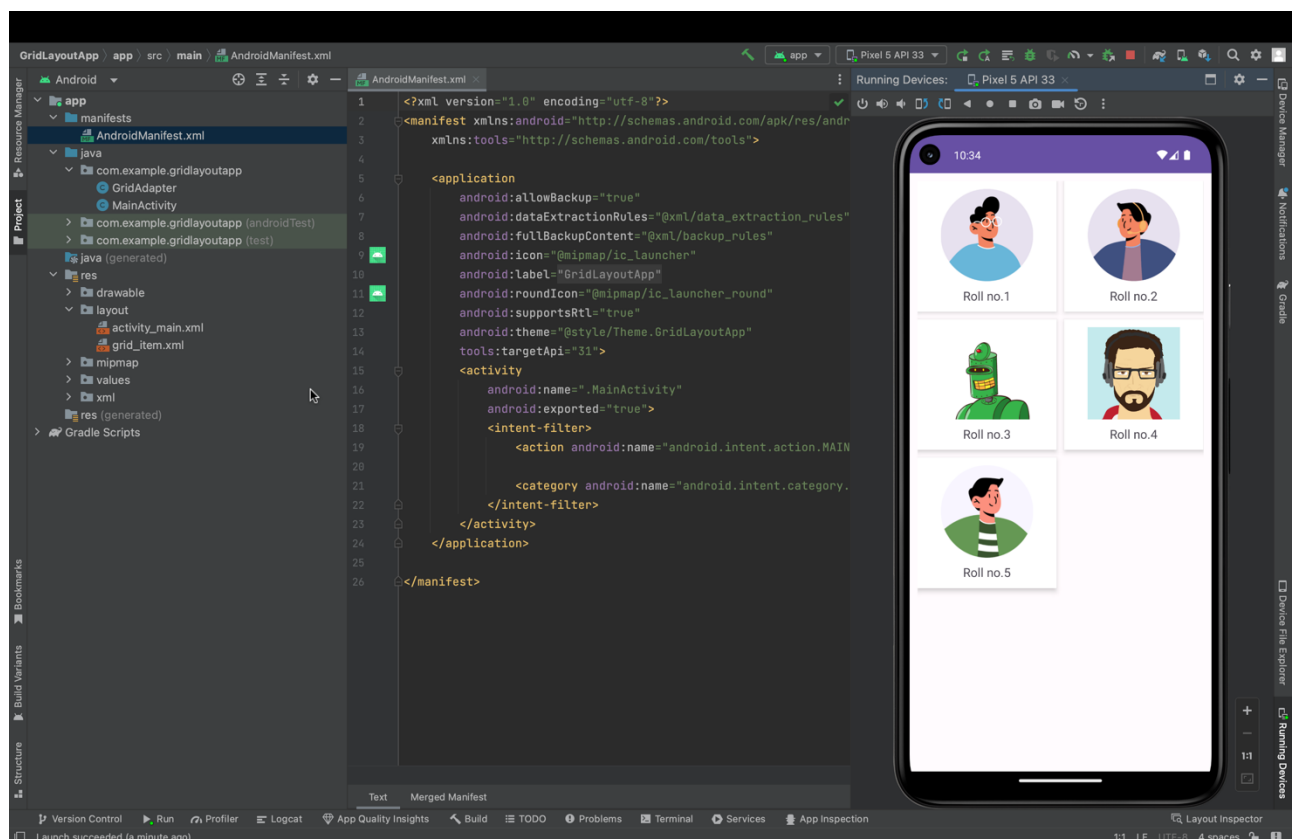


fig b. Android Studio Interface



PUNE INSTITUTE OF COMPUTER TECHNOLOGY, PUNE – 411043

Department of Electronics & Telecommunication Engineering

CONCLUSION:

.....

.....

.....